

Dmitry Muravyev

Delta encoding

Sprint 4: Mathematical foundation

Notation

Symbols used on several slides. Symbols used on one slide only are explained on that slide.

Symbol	Meaning
N	number of frames (pixels, samples)
i	frame number, $0 \le i < N$
C	number of channels
c	channel number, $0 \le c < C$
$x_{c,\,i}$	value of channel c in frame i
k_c	bits of channel c (target bit depth)
F	bits per frame, $F = \Sigma_c k_c$
b	bits of the block length field
q	side of the squares ($q = 1$: no squares)
δ	Δ option (which windows are used)

Symbol	Meaning
p	first frame of a block
l	block length in frames, $1 \le l \le l_{max}(p)$
$l_{max}(p)$	$\min(2^b,\ N - p)$
$m,\ w$	method; window width (bits)
$M_c,\ d_c$	methods of channel c; bits of its number
H	block header, $H = b + \Sigma_c d_c$
$L_w^F(p),\ L_w^A(p)$	longest Fixed / Adaptive block, width w
$\mathrm{cost}_c^*(p, l)$	cheapest channel c part of a block
$\mathrm{Bits}(p, l)$	size of the cheapest block $p \ldots p + l - 1$
$\mathrm{Best}(p)$	smallest size of frames $p \ldots N - 1$

All sizes are in bits. On the slides about one channel the index c is dropped: x_i, k.

A block is written $p \ldots p + l - 1$: it starts at frame p and has l frames.

Frames and squares

- A frame is the set of all channel values of one pixel (sample):
  - $x_i = (x_{0,i},\ x_{1,i},\ \ldots,\ x_{C-1,i})$, e.g. RGB 565: $k = (5, 6, 5)$, $F = 16$
- Splitting into $q \times q$ squares (q divides the image width W and height):
  - frame i is taken from pixel $\pi(i)$:

$$t = \lfloor i/q^2 \rfloor, \quad o = i \bmod q^2, \quad r = \lfloor t\,/\,(W/q) \rfloor, \quad u = t \bmod (W/q)$$

$$\pi(i) = (r \cdot q + \lfloor o/q \rfloor) \cdot W + u \cdot q + (o \bmod q)$$

The squares keep neighboring pixels close in the frame sequence, so the deltas are smaller.

$q = 1$ means no splitting: $\pi(i) = i$.

On this slide:

W	image width in pixels
$t,\ o$	number of the square; frame offset inside it
$r,\ u$	row and column of the square

0	1	4	5	8	9	12	13
2	3	6	7	10	11	14	15
16	17	20	21	24	25	28	29
18	19	22	23	26	27	30	31

Frame numbers for $q = 2$, $W = 8$

Block format

Block frames $l-1$	Method Channel 0	...	Method Channel $C-1$	Initial value Channel 0	...	Initial value Channel $C-1$	Data ...
b bits	d_0 bits		d_{C-1} bits	k_0 bits*		k_{C-1} bits*	

** the initial value is omitted for an uncompressed channel ($m = k_c$).*

$M_c = k_c + 1$ (one Δ option), $M_c = 2k_c$ (two Δ options); $d_c = \lfloor \log_2 (M_c - 1) \rfloor + 1$

$$H = b + \sum_{c=0}^{C-1} d_c$$

Channel bits k_c	**2**	**3**	**4**	**5**	**6**	**7**	**8**
d_c, one Δ option	2	2	3	3	3	3	4
d_c, two Δ options	2	3	3	4	4	4	4

The block length is stored as $l-1$,
so b bits give blocks of $1 \ldots 2^b$ frames.
Every block is self-contained: the decoder needs no state from the previous block.

Methods

m	Fixed window only	Adaptive window only	Fixed window first	Adaptive window first
0	Fixed, $w = 0$	Fixed, $w = 0$	Fixed, $w = 0$	Fixed, $w = 0$
$1 \ldots k-1$	Fixed, $w = m$	Adaptive, $w = m$	Fixed, $w = m$	Adaptive, $w = m$
k	Uncompressed	Uncompressed	Uncompressed	Uncompressed
$k+1 \ldots 2k-1$	—	—	Adaptive, $w = m - k$	Fixed, $w = m - k$

Cost of one channel in a block of l frames (bits, without the header), $k = k_c$:

Fixed window: $\mathrm{cost} = k + l \cdot w$ (initial value = minimum of the block)

Adaptive floating window: $\mathrm{cost} = k + (l-1) \cdot w$ (initial value = first frame)

Uncompressed: $\mathrm{cost} = l \cdot k$

All frames equal (Fixed, $w = 0$): $\mathrm{cost} = k$

The Δ option is a property of the whole stream: the compressor and the decoder (#define in Decompression.h) must use the same one.

Fixed window

The method with window width w can encode the frames $p \ldots p+l-1$ if

$$\max_{p \le i < p+l} x_i \; - \; \min_{p \le i < p+l} x_i \; \le \; 2^w - 1$$

Initial value $v = \min x_i$, data $x_i - v$ in w bits each.

$$L_w^F(p) = \max \left\{ l \le l_{max}(p) \; : \; \max_{p \le i < p+l} x_i - \min_{p \le i < p+l} x_i \le 2^w - 1 \right\}$$

	100	110	120	110	115	120	130	140	150	160	170	180	190	200	210	220
100	**0**	**10**	**20**	**10**	**15**	**20**	**30**	**40**	**50**	**60**	**70**	**80**	**90**	**100**	**110**	**120**

$w = 5$ (data $0 \ldots 31$), block length: 7

	103	114	118	107	102	131	119	128	127	115	110	130	100	101	122	125
100	**3**	**14**	**18**	**7**	**2**	**31**	**19**	**28**	**27**	**15**	**10**	**30**	**0**	**1**	**22**	**25**

$w = 5$, block length: 16

Example cost ($k = 8$, $w = 5$, $l = 16$): $8 + 16 \cdot 5 = 88$ bits instead of $16 \cdot 8 = 128$ bits.

On this slide:

v	initial value
x_i	frame values

Adaptive floating window

A window of width 2^w around the previous frame y, kept inside the value range $0 \ldots 2^k - 1$:

$$\mathrm{lo}_w(y) = \min\left(\max\left(y - 2^{w-1},\ 0\right),\ 2^k - 2^w\right)$$

Frame x_i fits if $\mathrm{lo}_w(x_{i-1}) \le x_i < \mathrm{lo}_w(x_{i-1}) + 2^w$; data $x_i - \mathrm{lo}_w(x_{i-1})$ in w bits.

$L_w^A(p) = \max\{\, l \le l_{max}(p) \ :\ \text{frames } p+1 \ldots p+l-1 \text{ fit} \,\}$, initial value x_p

	100	110	120	110	115	120	130	140	150	160	170	180	190	200	210	220
	100	26	26	6	21	21	26	26	26	26	26	26	26	26	26	26
lo:		84	94	104	94	99	104	114	124	134	144	154	164	174	184	194

$w = 5$, block length: 16. Cost: $8 + 15 \cdot 5 = 83$ bits.

The window follows the source data (the previous frame is the original value, the compression is lossless), so the position of the window does not depend on w. This is what makes the windows nested (Lemma 1). The clamping keeps the whole window inside the value range.

On this slide:

y	previous frame
$\mathrm{lo}_w(y)$	lowest value of the window

Block size and efficiency

A block $p \ldots p + l - 1$ with the methods $\mathbf{m} = (m_0, \ldots, m_{C-1})$ takes

$$\mathrm{Bits}(p, l, \mathbf{m}) = H + \sum_{c=0}^{C-1} \mathrm{cost}_c(m_c,\ l)$$

Method m_c may be used only if it lasts long enough: $l \le L_{c,\,m_c}(p)$.

$$E_{block} = \left(1 - \frac{\mathrm{Bits}(p, l, \mathbf{m})}{l \cdot F}\right) \cdot 100\%, \qquad E_{overall} = \left(1 - \frac{\Sigma_j \mathrm{Bits}_j}{N \cdot F}\right) \cdot 100\%$$

The output is a chain of blocks $j = 0, 1, \ldots$ covering all N frames, padded to whole bytes:

$$\mathrm{bytes} = \left\lceil \frac{1}{8} \sum_j \mathrm{Bits}_j \right\rceil$$

Max E for $k = 8$: the data part of a block can't be smaller than w/k of the raw data.

w	**max** E
0	100.0%
1	87.5%
2	75.0%
3	62.5%
4	50.0%
5	37.5%
6	25.0%
7	12.5%
8	0.0%

On this slide:

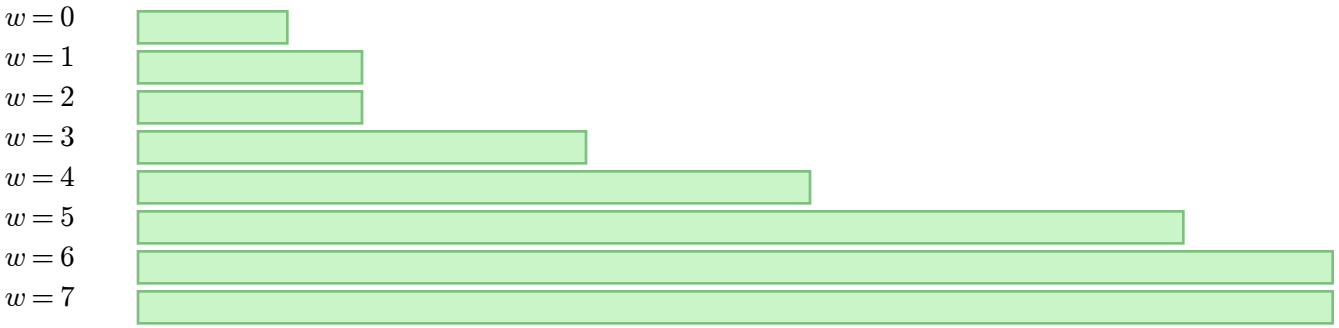
$\mathbf{m}$	the methods of all channels in the block
$L_{c,\,m}(p)$	longest block for method m: $L_w^F(p)$ or $L_w^A(p)$ (or $l_{max}(p)$)
$E,\ \mathrm{Bits}_j$	compression efficiency; size of block j of the chain

Lemma 1: nested windows

For every start frame p and channel:

$$L_0^F(p) \le L_1^F(p) \le \ldots \le L_{k-1}^F(p), \qquad L_1^A(p) \le L_2^A(p) \le \ldots \le L_{k-1}^A(p)$$

- Fixed: the condition $\mathrm{range} \le 2^w - 1$ only gets weaker as w grows.
- Adaptive: for the same previous frame y the window of width 2^{w+1} contains the window of width 2^w:
  - $\mathrm{lo}_{w+1}(y) \le \mathrm{lo}_w(y)$ and $\mathrm{lo}_w(y) + 2^w \le \mathrm{lo}_{w+1}(y) + 2^{w+1}$;
  - without clamping: $y - 2^w \le y - 2^{w-1}$ and $y + 2^{w-1} \le y + 2^w$;
  - clamping at 0 or $2^k - 2^w$ moves both windows the same way (case analysis,
  - also checked exhaustively for all y, w and $k \le 12$).
- The previous frame is the source value, the same for all w, so the nesting holds frame by frame.



Consequence: one scan of the data finds the lengths of all windows at once (the break points of $w = 0, 1, 2, \ldots$ in turn).
$O(2^b + k)$ instead of $O(k \cdot 2^b)$ per channel.

Lemma 2: prefixes and the cheapest method

- If a method works for l frames from p, it works for any shorter block from p:
  - Fixed: a prefix has a smaller (or equal) range; Adaptive: a prefix has fewer frames to fit.
- So in a block of exactly l frames channel c may use every method with $L_{c,\,m}(p) \geq l$.
- For the same l a wider window only costs more, so the best choice in each family is the
  - narrowest window that lasts long enough:

$$w^F(l) = \min\left\{w:\ L_w^F(p) \geq l\right\}, \qquad w^A(l) = \min\left\{w:\ L_w^A(p) \geq l\right\}$$

$$\text{cost}_c^*\,(p, l) = \min\left(l \cdot k,\quad k + l \cdot w^F(l),\quad k + (l-1) \cdot w^A(l)\right)$$

$$\text{Bits}(p, l) = H + \sum_{c=0}^{C-1} \text{cost}_c^*\,(p, l)$$

$\text{Bits}(p, l)$ is the exact size of the cheapest block of l frames starting at p: every decodable block of that length is at least this big, and the narrowest windows give a real block of exactly this size.

On this slide:

$w^F(l)$	narrowest Fixed window
$w^A(l)$	narrowest Adaptive window
	(lasting $\geq l$ frames; $k = k_c$)

Frame-by-frame mode: exact best-block search

Sprints 2–3: a block is a combination of methods $\mathbf{m}$, its length is $l = \min_c L_{c,\,m_c}(p)$,
and the best block has the highest E_{block}. Brute force tries all $\Pi_c M_c$ combinations.

Group the combinations by their length. For a candidate length l (one of the lengths $L_{c,\,m}(p)$):

$$\mathrm{cost}_c^{*}(p,l) = \min_{m:\ L_{c,\,m}(p) \ge l} \mathrm{cost}_c(m,l), \qquad \mathrm{cost}_c^{=}(p,l) = \min_{m:\ L_{c,\,m}(p) = l} \mathrm{cost}_c(m,l)$$

$$\mathrm{Bits}^{=}(p,l) = H + \sum_c \mathrm{cost}_c^{*}(p,l) + \min_j \left(\mathrm{cost}_j^{=}(p,l) - \mathrm{cost}_j^{*}(p,l)\right)$$

$\mathrm{cost}_c^{=}(p,l)$: the cheapest method of channel c that ends exactly at l; j: a channel that ends at l.

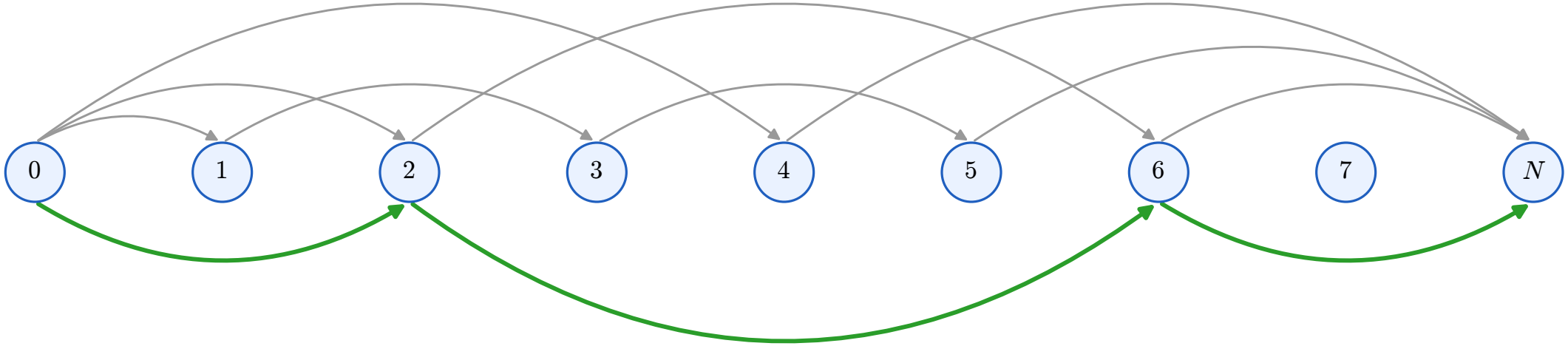
- Proof: the length is exactly l if every channel lasts $\ge l$ and at least one channel j ends at l.
  - For a fixed j the best is $\mathrm{cost}_j^{=} + \Sigma_{c \ne j} \mathrm{cost}_c^{*}$; minimizing over j gives the formula.
- Best block: the largest $1 - \mathrm{Bits}^{=}(p,l) \,/\, (l \cdot F)$ over at most $\Sigma_c M_c$ candidate lengths.
- Ties: the lexicographically first $\mathbf{m}$, rebuilt channel by channel, so the result is the same
  - block as the brute force, byte for byte.

Complexity: $O\big((\Sigma_c M_c)^2\big)$ instead of $O(\Pi_c M_c)$.
Logo RGB 888: 17.5 s → 0.19 s, same output.

Used only by the library's frame-by-frame builder. The optimal chain (next slides) chooses l itself: every channel just takes $\mathrm{cost}_c^{*}(p,l)$, and no channel has to end exactly at l.

Frame format	Brute force, 2 Δ	Exact, 2 Δ ($\le$)
RGB 565	1 200	$34^2 = 1\,156$
RGB 888	4 096	$48^2 = 2\,304$
RGB 48 bit	32 768	$96^2 = 9\,216$
4 ch. $\times$ 32 bit	16 777 216	$256^2 = 65\,536$

Chain of blocks as a shortest path



Frame boundaries are nodes $0 \ldots N$. A block $p \ldots p + l - 1$ is an edge $p \to p + l$ of weight $\mathrm{Bits}(p, l)$.

Every compressed stream is a path from 0 to N, and its size is the weight of the path.

$$\mathrm{Best}(N) = 0$$

$$\mathrm{Best}(p) = \min_{1 \le l \le l_{max}\,(p)} (\mathrm{Bits}(p, l) + \mathrm{Best}(p + l))$$

$$\mathrm{size}_{min} = \mathrm{Best}(0)$$

$\mathrm{Best}(p)$ is computed from $p = N - 1$ down to 0; the chosen length of every position is stored,
and the chain is restored from frame 0 forward.

Theorem: the chain is optimal

For the given b, δ and q, the restored chain has the minimum size among all streams that the decoder accepts.

Proof

- Any accepted stream is a sequence of blocks with lengths $1 \le l_j \le 2^b$ and $\Sigma_j l_j = N$.
- In each block every channel uses a method that lasts long enough, so the block is at least $\mathrm{Bits}(p, l)$ (Lemma 2).
- By induction from $p = N$ down to 0, a stream for frames $p \ldots N-1$ is at least $\mathrm{Best}(p)$.
- The restored chain uses, in every block, the narrowest windows that reach $\mathrm{Bits}(p, l)$, with the initial
  - value = minimum (Fixed) or first frame (Adaptive), so its size is exactly $\mathrm{Best}(0)$. ■

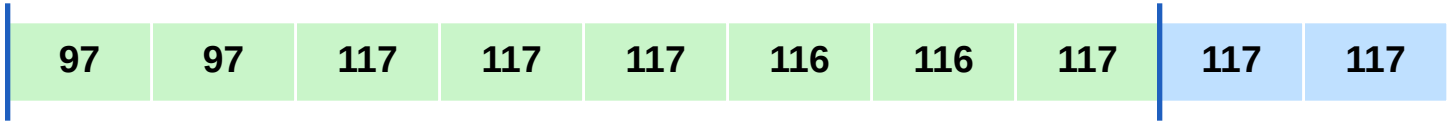
Optimal "per block" (sprints 2–3: the most efficient block at each step, or the best block for every frame) is not optimal for the chain: a very efficient long block can leave a tail that compresses badly, and a short, less efficient block can open the way for a much better one. The example is on the next slide. $\mathrm{Bits}(p, l)$ does not depend on the previous blocks, so the dynamic programming covers all chains.

l_j is the length of block j of the stream.

Example: greedy vs optimal chain

$k = 8$, $b = 3$ (blocks of $1 \ldots 8$ frames), Fixed window first ($d = 4$), $H = 7$ bits, raw data 80 bits.

Greedy: the most efficient block at each step

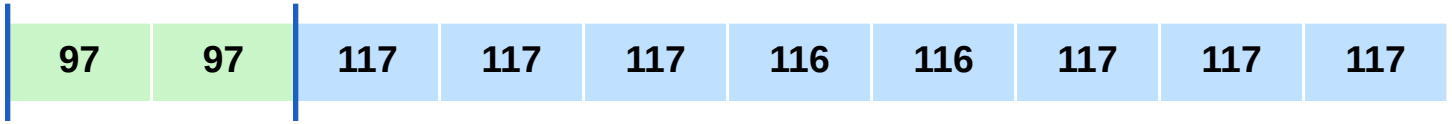


70 bits
E = 12.5%

Fixed $w = 5$: 55 bits, E=14.06%

Fixed $w = 0$: 15 bits, E=6.25%

Optimal chain (dynamic programming)



38 bits
E = 52.5%

Fixed $w = 0$: 15 bits, E=6.25%

Fixed $w = 1$: 23 bits, E=64.06%

Greedy: $(7 + 8 + 8 \cdot 5) + (7 + 8) = 55 + 15 = 70$ bits; the first block has $E = 14.1\%$.

Optimal: $(7 + 8) + (7 + 8 + 8 \cdot 1) = 15 + 23 = 38$ bits, 46% smaller; the first block has only $E = 6.25\%$.

$\mathrm{Bits}(p, l)$ is piecewise linear

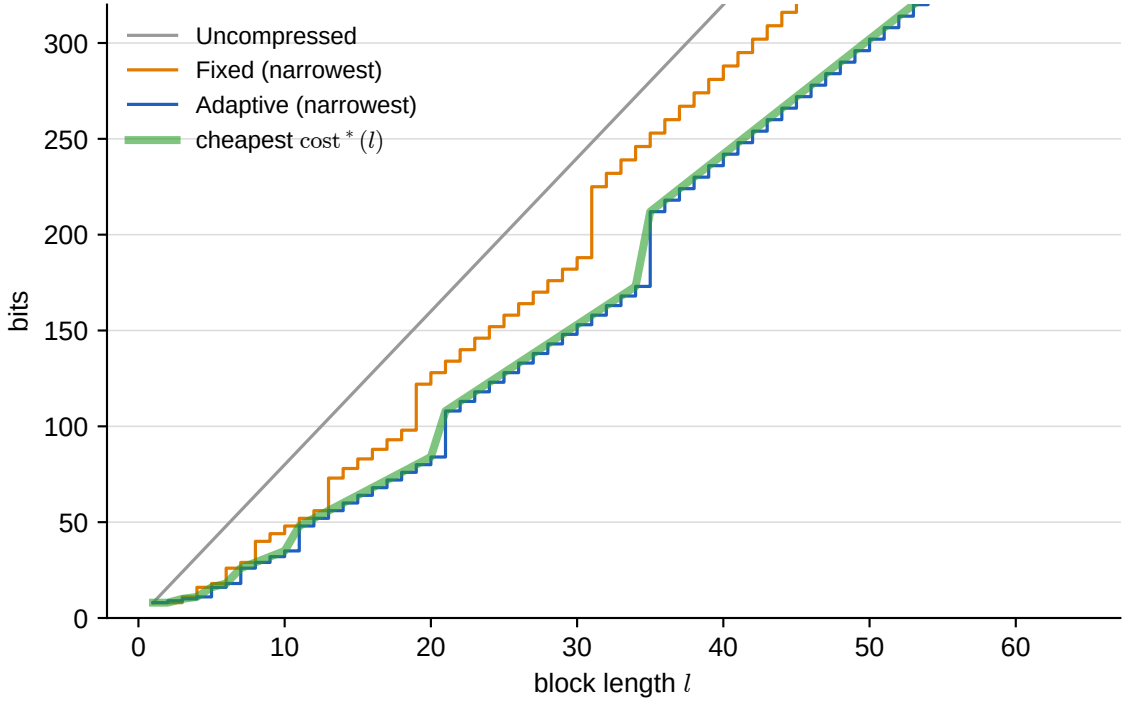
For one channel ($k = k_c$) the candidates are three lines $\alpha + \sigma \cdot l$ (the narrowest windows lasting l frames):

Uncompressed: $0 + k \cdot l$, Fixed: $k + w^F \cdot l$, Adaptive: $(k - w^A) + w^A \cdot l$

- w^F and w^A change only after l passes $L_w^F(p)$ or $L_w^A(p)$. Between those points the cheapest
  - line $\alpha + \sigma l$ stays the cheapest until a line i with a smaller slope $\sigma_i < \sigma$ becomes cheaper,
  - i.e. $\alpha_i + \sigma_i l < \alpha + \sigma l$, from the first whole l above $(\alpha_i - \alpha)/(\sigma - \sigma_i)$:

$$l_\times = \left\lfloor \frac{\alpha_i - \alpha}{\sigma - \sigma_i} \right\rfloor + 1$$

So the block costs $\mathrm{Bits}(p, l) = \alpha_{blk} + \sigma_{blk} \cdot l$ on segments of l. A segment ends at the nearest $l_\times - 1$ or $L_w^F(p)$, $L_w^A(p)$ over all channels. Inside a segment the next length costs one addition: $+\sigma_{blk}$.



On this slide:

$\alpha,\ \sigma$	line of a candidate: fixed part, bits/frame
$\alpha_i,\ \sigma_i$	another candidate
$l_\times$	where it gets cheaper
α_{blk}	$H + \Sigma_c \alpha_c$
σ_{blk}	$\Sigma_c \sigma_c$

Lemma 3: pruning

$\mathrm{cost}_c^*(p, l)$ never decreases as l grows:

- every candidate line has a non-negative slope and the windows only get wider;
- the set of methods that last long enough only shrinks (Lemma 2).

So $\mathrm{Bits}(p, l)$ is non-decreasing in l, and since $\mathrm{Best}(\cdot) \geq 0$:

$$\mathrm{Bits}(p, l) \geq T \Rightarrow \mathrm{Bits}(p, l') + \mathrm{Best}(p + l') \geq T \quad \text{for all } l' \geq l$$

The scan of lengths stops at the first such l. No candidate is lost, so the result stays optimal.

Also $\mathrm{Best}(p + l)$ never grows with l (dropping the first frame of a block never makes it longer), so a whole range of lengths $l \ldots l_2$ is skipped if $\mathrm{Bits}(p, l) + \mathrm{Best}(p + l_2) \geq T$ (chunks of 256 and 8 lengths).

In practice a block that is already bigger than the best whole tail appears after a few dozen frames, so the scan looks at a small part of the 2^b possible lengths. Example: mit-text, $b = 15$ (32 768 lengths), takes about 1 second for the whole file.

On this slide:

T	the best total found so far for p
l', l_2	longer lengths

Incremental lengths (backward pass)

The DP goes from $p = N - 1$ down to 0, so the lengths at p are updated from the ones at $p + 1$:

Fixed window

- End of the longest block: $e_w(p) = p + L_w^F(p)$. If $p \ldots e - 1$ fits, $p + 1 \ldots e - 1$ fits too, so $e_w(p) \leq e_w(p + 1)$.
- Start from $e = e_w(p + 1)$ and move it left while $e > p + 2^b$ or the range of $x_p \ldots x_{e-1}$ is $> 2^w - 1$.
- The min and max of the block are kept in two monotonic deques (amortized O(1)).

Adaptive floating window

- A block breaks at the first frame that doesn't fit the window around its predecessor:

$$\beta_w(p) = p + 1 \text{ if } x_{p+1} \text{ doesn't fit around } x_p, \text{ otherwise } \beta_w(p) = \beta_w(p + 1)$$

$$L_w^A(p) = \min\left(\beta_w(p) - p,\ l_{max}(p)\right)$$

All lengths of all windows for all positions: $O(N \cdot \Sigma_c k_c)$ amortized,
instead of rescanning up to 2^b frames for each window at each position.

On this slide:

$e_w(p)$	end of the longest Fixed block (the first frame after it)
$\beta_w(p)$	first frame that breaks the Adaptive block

Complexity

Step	Before (sprint 3)	Now
Lengths of all methods at p	$O(\Sigma_c M_c \cdot 2^b)$	$O(\Sigma_c k_c)$ amortized
Best block at p (frame-by-frame mode)	$O(\Pi_c M_c)$	$O((\Sigma_c M_c)^2)$
Chain of blocks	heuristic (frame-by-frame)	exact DP, pruned
Memory allocated, Logo RGB 888	41 GB (total)	< 30 MB
Compression time, all test cases	53.7 s	2.9 s
Decoder (flash / RAM)	—	unchanged

Compressor implementation: data cached once as arrays, Span<T> and stackalloc in the hot loops,
no allocations per frame, multithreaded search for the legacy chain and for Auto settings.

Auto settings

The DP gives the exact output size without writing the stream, so the settings can be searched:

$$(q^*,\ \delta^*,\ b^*) = \arg \min_{q \in Q,\ \delta \in D,\ b \in B} \mathrm{Best}_{q,\delta,b}(0)$$

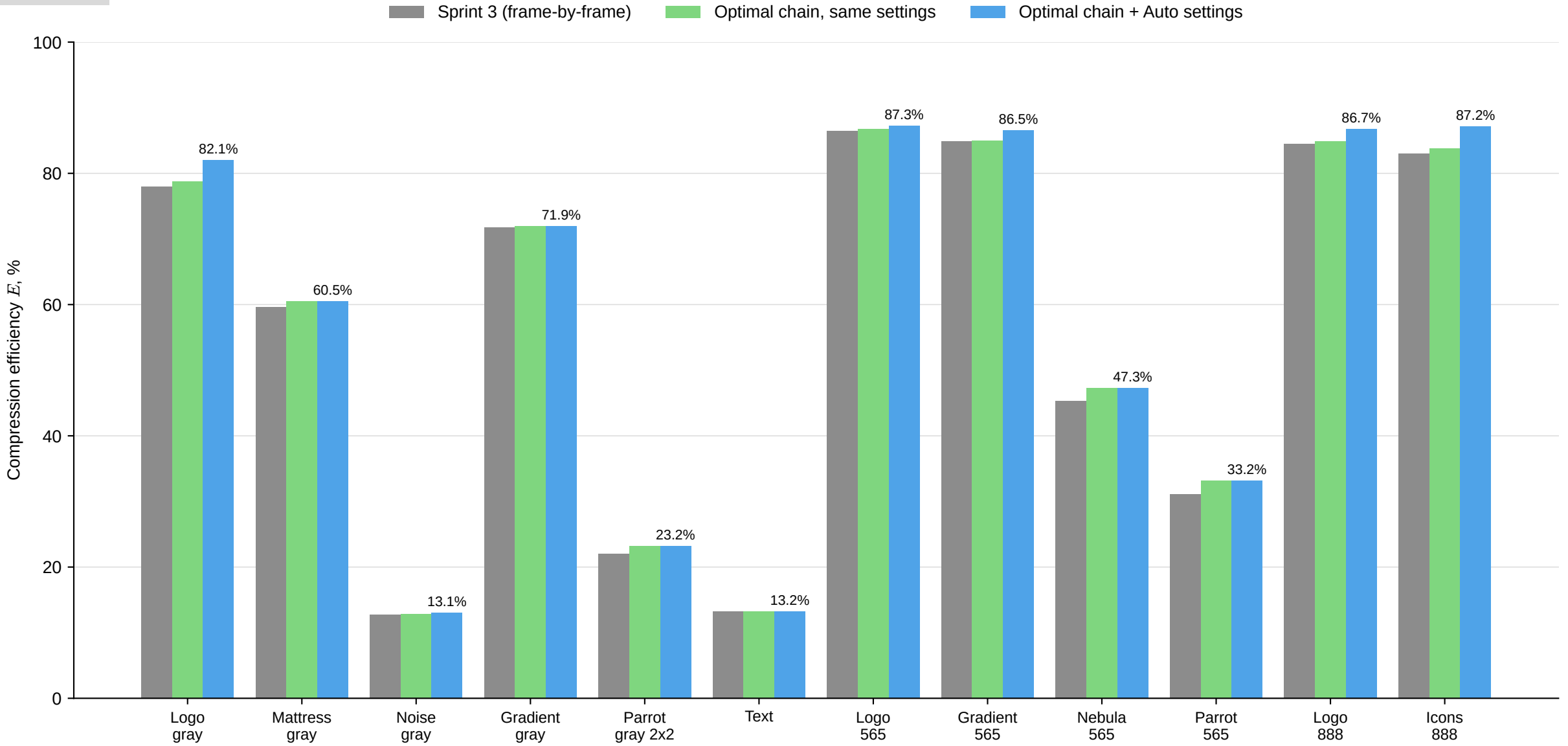
- All (q, δ) pairs are tried in parallel: squares and Δ option interact,
  - choosing them one after another was up to 36% worse.
- For each pair b grows until the size gets worse 3 times in a row (in practice the
  - size has a single minimum in b; this stop rule is the only heuristic step).
- Several b share one pass: the lengths do not depend on b (only l_{max} does).
- The predicted size equals the real output, so the choice is not an estimate.

Auto may choose any Δ option. The decoder must be built with the matching #define in Decompression.h (DECOMPRESSION_FIXED_WINDOW_FIRST etc.).

On this slide:

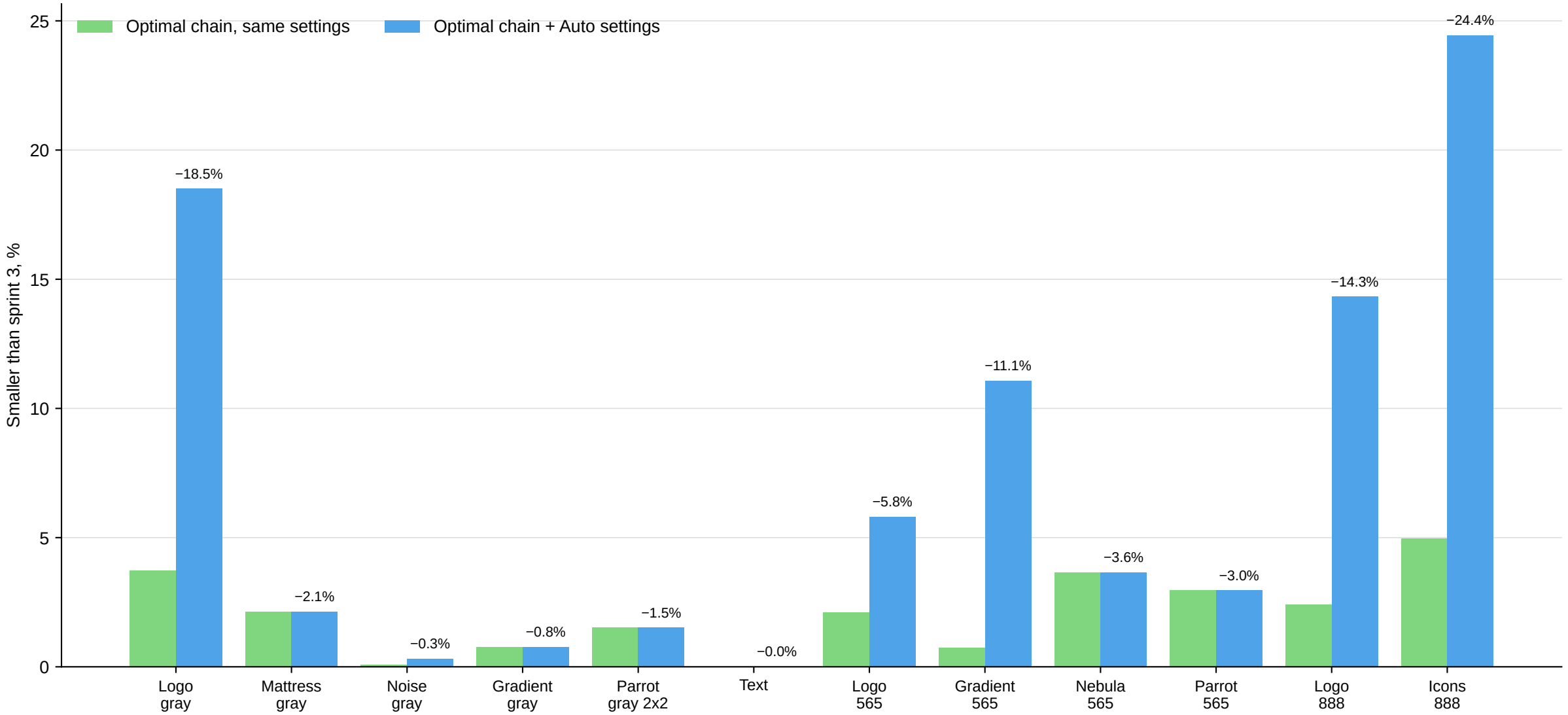
Q	sides that divide the image width and height (1: none)
D	the 4 Δ options
B	the offered values of b
$\mathrm{Best}_{q,\delta,b}$	Best for these settings

Results



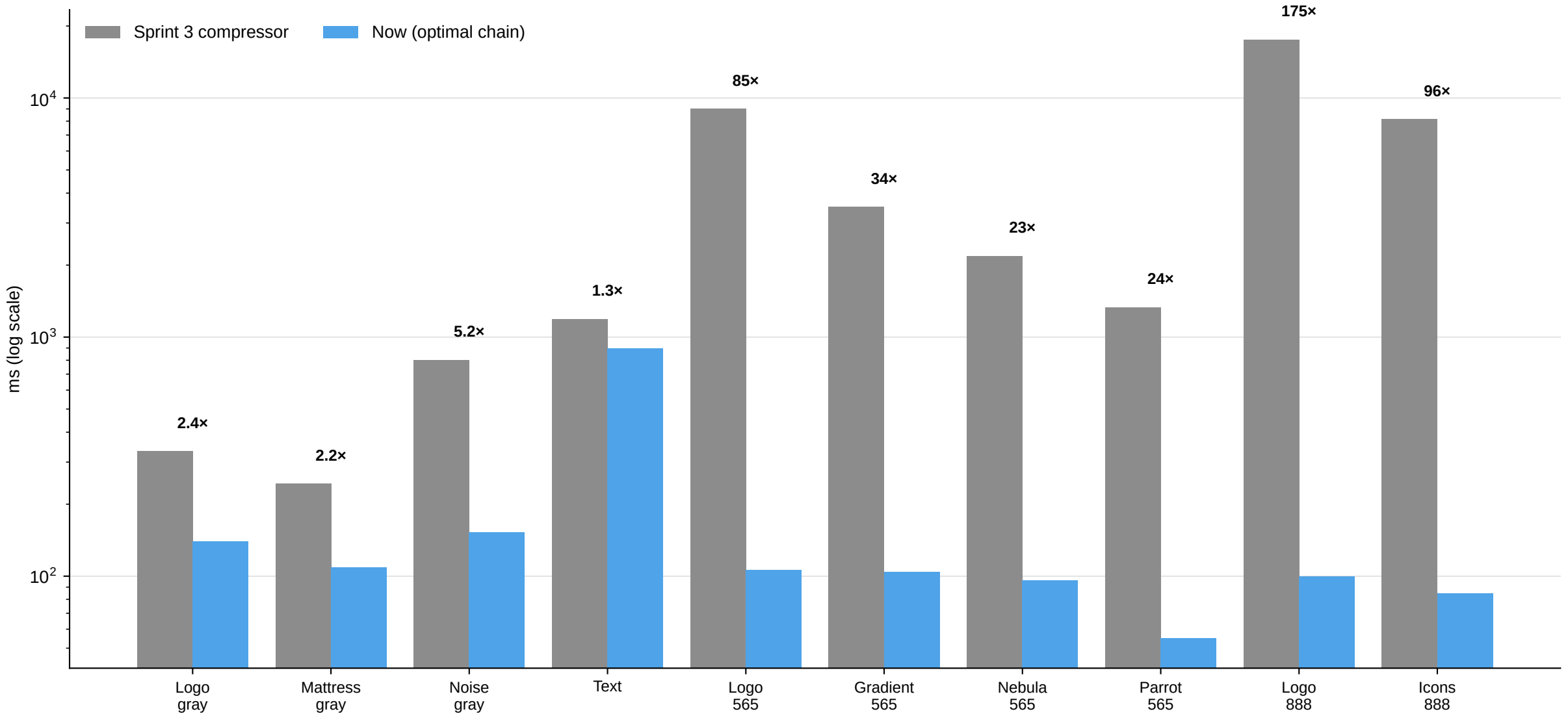
Sprint 3 settings for the first two series; 240 × 240 images, text 32 KB. Every output is checked with the C++ decoder.

Results: size reduction



Auto: b, Δ option and squares chosen per image (1–13 s per image on 4 cores).

Results: compression time



Same settings, one image at a time, 4-core cloud machine, .NET 10.

Decompression

- The stream format is unchanged: the existing decoder reads every new file.
- All the improvements are on the compressor side; the decoder stays as small as before:
  - per channel it keeps only the method, the window and the last value;
  - one pass over the data, no tables, no buffers beyond one block header.
- The Δ option and the channel bit depths must match the decoder build (#define).
- Squares need DECOMPRESSION_USE_SQUARES and the same square side.

Next sprint

- Cross-platform interface (Avalonia) on top of the UI-free core library.
- Repository cleanup, build and test automation, a proper release.
- Format extensions worth measuring (would change the decoder):
  - a flag "same methods as the previous block" to shorten headers;
  - variable-size length field for long uniform blocks.